

Algorithm Analysis: Measuring and Predicting Running Times – Student Guide

Introduction

The **3-Sum** problem asks whether an integer array contains triples of values whose sum is zero. In this project, each implementation's `count` method returns the number of triples that sum to zero. You will measure three implementations, compare how their running times grow as input size increases, and use those measurements to make predictions.

Background: Measuring and Predicting Running Time

Stopwatch and fair measurements

Construct a `Stopwatch` immediately before the operation you want to measure, then call `elapsedTime()` afterward to get elapsed time in seconds. Measure the algorithm—not input-file reading or input generation—so the results better reflect the algorithm's cost.

For each test, read the file into an `int` array before starting the stopwatch. Start timing immediately before calling `count()` and stop immediately after it returns. Use the same input files for all three phases.

Doubling ratios

A doubling ratio compares the running time for an input of size $2N$ with the time for size N :

$$\text{doubling ratio} = \frac{\text{time}(2N)}{\text{time}(N)}$$

If the ratio is approximately 2^b , the measured growth suggests an approximate order of $O(n^b)$. This is an empirical estimate, not a proof. Actual ratios may differ from theoretical expectations because of overhead, machine load, input distribution, and timer resolution.

Order of growth	Approximate ratio when N doubles
$O(n)$	2
$O(n \log n)$	Slightly above 2
$O(n^2)$	4
$O(n^2 \log n)$	Slightly above 4
$O(n^3)$	8

For $O(n^2 \log n)$, the ratio when N doubles is $4 \frac{\log(2N)}{\log(N)}$, which is slightly above 4. The ratio approaches 4 as N grows, but is not exactly 4 for finite N .

Log-log plots

On a log-log plot, use $x = \log(N)$ and $y = \log(\text{time})$. For a power-law trend, the slope b indicates approximate growth of $O(n^b)$. Timing noise can make measurements uneven, and very small or zero measurements are not useful for a log plot. Use meaningful positive times, consistent inputs, and the same environment when possible.

Project Setup and Fair Testing

1. Compile all project files.
2. Complete the supplied skeleton main methods where TODOs ask you to read an input file, call `count()`, and print the result. Keep input-file reading outside the timed interval.
3. Use the Provided input txt files across all three phases. For each run, read the selected file into an `int` array first, then time only the call to `count()`.
4. If measurements are too small to be reliable, use sufficient timing precision or repeat runs and report a reasonable representative measurement. Do not report zero as a meaningful running time.

For each table, compute a doubling ratio as the current row's time divided by the previous row's time. The first row has no ratio. Mark predictions clearly; do not present predictions as measurements.

Phase 1 – Brute Force ThreeSum

1. Complete `ThreeSum.count()` with three nested loops that examine each distinct triple once, such as indices satisfying $i < j < k$. Increment the count when `a[i] + a[j] + a[k] == 0`.
2. Complete the main TODO to read a selected input file, call `count()`, and print the result. Keep file reading outside the timed interval.
3. Run the algorithm on 1K, 2K, 4K, 8K, 16K, and 32K inputs. Record each time and calculate ratios for successive doubling pairs.
4. Predict the running times for 64K and 128K from your measured trend. Brute-force estimates at these sizes may become impractical; label them as extrapolations rather than running the program if the cost is unreasonable.
5. Written response: What ratio did you observe, and what order of growth does it suggest?

The ratio was somewhat inconsistent, but the average was around 8x. Given N was increasing by 2x on each iteration, a ratio of 8x suggests that the order of growth is $O(N^3)$.

N	Time (s)	Doubling Ratio	Predicted?
1,000	0.173	N/A	No
2,000	1.27	7.34	No
4,000	5.57	4.39	No
8,000	69.955	12.57	No
16,000	560	8.00	No
32,000	4522	8.075	Yes — extrapolation
64,000	36,515.15	8.075	Yes — extrapolation
128,000	294,859.836	8.075	Yes — extrapolation

For the brute-force version, the three loops examine a number of triples that grows on the order of n^3 . Compare your measured doubling ratios with the expected ratio of about 8 for cubic growth.

Phase 2 – ThreeSumFast (Arrays.sort + Binary Search)

1. In `ThreeSumFast.count()`, complete the `BinarySearch.indexOf` lookup for `-a[i]-a[j]`. Increment the count only when the returned index is greater than `j`. This ensures the third element has a distinct, later index and that each triple is counted once.
2. Complete the main input-file TODO. Exclude input reading from the timed interval.
3. Run the same six input sizes, record times, calculate successive doubling ratios, and predict 64K and 128K.
4. Written response: How does this phase compare with Phase 1, and why is it faster? Discuss the measured ratios and the expected growth.

This phase is much faster than Phase 1, with a ratio of about $\frac{1}{2}x$ the amount of phase 1. It is faster because the sorted array only requires $\log(N)$ time – the main time is taken by sorting the array, which is $O(N\log N)$

N	Time (s)	Doubling Ratio	Predicted?
1,000	0.018	N/A	No
2,000	0.063	3.5	No
4,000	0.315	5	No
8,000	1.34	4.25	No
16,000	4.616	3.44	No
32,000	18.763	4.06	No
64,000	75.99	4.05	Yes — extrapolation
128,000	307.76	4.05	Yes — extrapolation

The algorithm sorts the array, considers $O(n^2)$ pairs, and performs a binary search taking $O(\log n)$ for each pair. Java's `Arrays.sort(int[])` is typically $O(n \log n)$, so the pair-and-search work dominates and the overall running time is $O(n^2 \log n)$. Its doubling ratios are asymptotically slightly above 4.

Phase 3 – ThreeSumInsertionSort

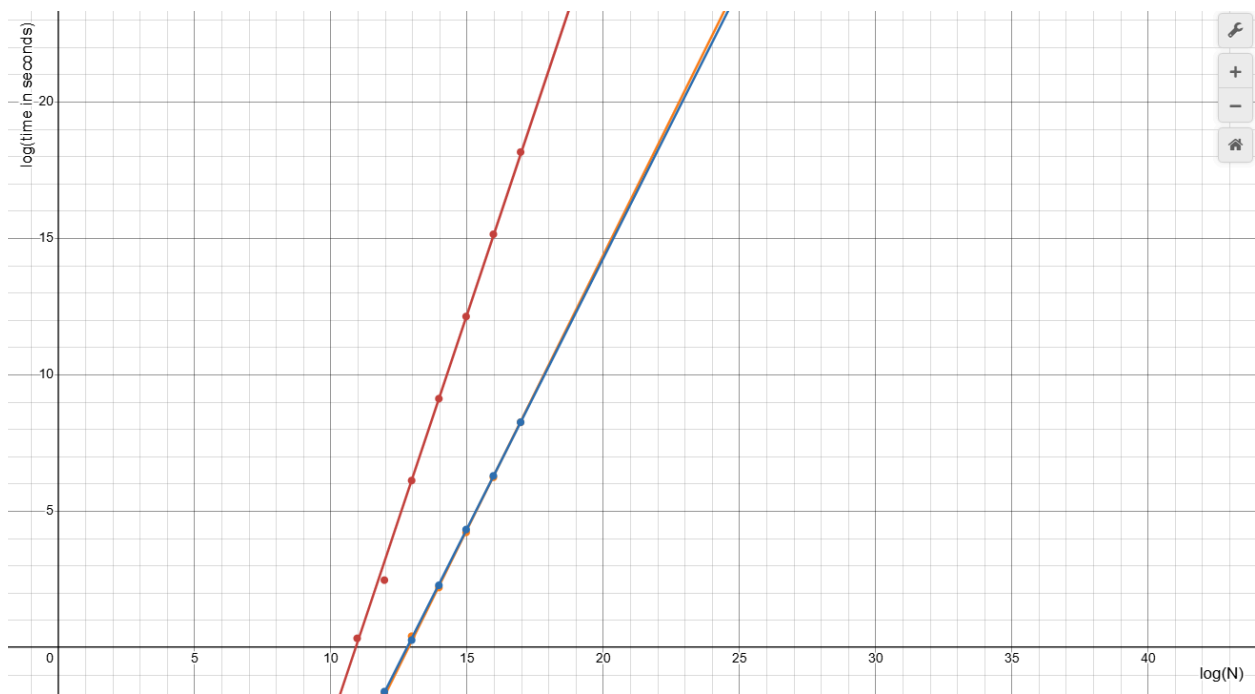
1. In the insertion-sort version, use `insertionSort(a)` instead of `Arrays.sort(a)`.
2. Complete the same binary-search lookup for `-a[i]-a[j]` and increment the count only when the returned index is greater than `j`. Complete the main TODO as well, keeping input reading outside the timed interval.
3. Run the same six input sizes and record ratios. Predict 64K and 128K only if feasible, and clearly label predictions as extrapolations.
4. Written response: Compare this phase with Phase 2. What effect does changing the sort have on measured time and ratios?

This phase was slightly slower than Phase 2, but the time and ratio did not change too drastically. Insertion sort is $O(N^2)$, but the binary search stays the same complexity. Since $O(N^2)$ explodes much faster than $O(N \log N)$, this should be slower than phase 2, as we see (although not to the extent theoretically).

N	Time (s)	Doubling Ratio	Predicted?
1,000	0.023	N/A	No
2,000	0.071	3.08	No
4,000	0.328	4.62	No
8,000	1.213	3.69	No
16,000	4.887	4.03	No
32,000	20.158	4.12	No
64,000	78.78	3.908	Yes — extrapolation, if feasible
128,000	307.86	3.908	Yes — extrapolation, if feasible

Log-Log Plot

1. Plot measured N and time on log-log axes, by hand or with a tool. Use only measured, positive times.
2. Label the axes $\log(N)$ and $\log(\text{time in seconds})$. Show each implementation as a separate series or use separate plots.
3. Draw or calculate a best-fit line for each implementation and estimate its slope.
4. Compare each slope with your doubling-ratio estimate. Explain whether the plot confirms or qualifies your estimate, noting any noisy or unusually small measurements.
5. You may extend a fitted trend to estimate 64K and 128K, but identify those values as predictions and do not overclaim precision.



The red, orange, and blue lines correspond to phases 1, 2, and 3, respectively. The last two points on all lines are estimates. The slopes for phases 2 and 3 are about ~ 2 (2.02 and 1.98 respectively), which conforms to the ratio as this is a log curve ($2^2 = 4$, as calculated), while the slope for phase 3 was about ~ 3 (2.97), which conforms to the ratio as $2^3 = 8$.

Analysis Questions

Answer all five questions in complete sentences. What doubling ratio did you observe for each implementation? What order of growth does that suggest?

1. How did your log-log plot help you make predictions?

Without the log plot, the x-axis (N) would dominate heavily, leading to an almost-flat looking line with extremely small slope due to the relatively small times. The log scale helped amplify the difference and visualize the linear relationship much better.

2. Why does replacing Arrays.sort with insertion sort change the overall complexity of ThreeSumFast?

The time complexity of both algorithms are different—Arrays.sort uses quicksort which is an $O(N \log N)$ algorithm, as opposed to the insertion sort which is an $O(N^2)$ algorithm—which results in different overall complexities of the algorithms.

3. If you had to process 1 million integers, which algorithm would you choose and why?

I would choose `Arrays.sort` due to its lower complexity (quicksort, $O(N \log N)$).

4. What is the relationship between the sorting step and the binary search step in `ThreeSumFast`?

The sorting step overshadows the binary search step—the sorting step contains a minimum of $O(N \log N)$ and maximum of $O(N^2)$ depending on the algorithm, which is vastly greater than the $O(\log N)$ complexity of the search step.